SMART INDIA HACKATHON 2026 · PROBLEM STATEMENT SIH26085 ·
MINISTRY OF EARTH SCIENCES · DISASTER MANAGEMENT · SOFTWARE

VARUNA

Street-Level Urban Flood Nowcasting Digital Twin

Vector-graph Assimilated Runoff & Urban Nowcasting Architecture

"*Every street. Three hours early.*"

The Legendary Blueprint — verdict on the problem statement, the winning idea, physics and mathematics, data playbook for India, full technology stack and languages, architecture, API design, 36-hour build plan, demo script, judge Q&A, risk register, business case, and the roadmap from V1 to V100.

Prepared for: SIH 2026 team, VIT · **Date:** 2 September 2026 · **Document version:** 1.0
Sponsor reading of the PS: the ministry asked for four concrete deliverables — (1) a radar-to-surface routing pipeline, (2) a directed-graph drainage model with capacity/surcharge prediction, (3) a real-time street-by-street GIS dashboard with 0–3 h depth projections in centimetres, (4) a flood-safe routing API. VARUNA delivers all four and adds the parts nobody else will bring.

Contents

0. How to read this document

You asked for a plan at "version 100" level. A blueprint that only describes the final vision is useless in a 36-hour finale, and a blueprint that only describes an MVP will not amaze anyone. So every capability in this document is tagged with the level at which it must exist:

Tag	Meaning	Rule
V1	Must work end-to-end at the SIH grand finale	If it is not V1, it does not get built during the 36 hours. V1 is what judges touch.
V10	Pilot-grade, 6–12 months with a city partner	Designed for now (interfaces, schemas, placeholders), built later. You talk about it in the pitch with a concrete plan.
V100	The national/global vision	The reason a ministry, an insurer or an investor cares. Keep it credible by anchoring it to V1 components.

Read Sections 1–4 first (the verdict and the idea), then Section 5 (architecture), then Section 11 (how you actually build it). Sections 6–10 are the reference material your engineers will live in. Sections 13–14 are what your presenters must memorise.

1. Verdict: is SIH26085 a good problem statement?

> **Short answer: yes — it is one of the strongest software problem statements in the entire SIH 2026 catalogue, and it is worth the risk.** It is sponsored by the Ministry of Earth Sciences (the parent of IMD, IITM, NCMRWF, NCCR and INCOIS), it sits in the Disaster Management theme, it has a live multi-thousand-crore national procurement programme behind it, and it is technically rich enough that a great team can build something that no other team in the hall can replicate. It is also hard, and most teams will fail it in a predictable way — which is exactly why it is winnable.

1.1 Scorecard

Dimension	Score	Reasoning
Real-world impact	**10/10**	Annual loss of life, gridlock and economic damage in Mumbai, Delhi, Chennai, Bengaluru, Hyderabad. Street-level, 0–3 h warnings are the missing operational layer between weather forecasts and emergency action.
Demand & procurement path	**9/10**	The Centre has approved the Urban Flood Risk Management Programme for 18 cities (Phase-I: 7 metros, ₹3,075.65 crore; Phase-II: 11 cities, ₹2,444.42 crore) and the programme explicitly funds flood early-warning and real-time data systems, not only concrete drains. MoES itself operates IFLOWS-Mumbai and C-FLOWS-Chennai and wants the next layer.
Technical richness	**10/10**	Radar meteorology + 2D hydrodynamics + graph hydraulics + data assimilation + GNN surrogates + GIS + time-dependent routing. Enough surface area for six engineers to each own a genuine engine.
Uniqueness ceiling	**9/10**	The PS forces a coupled system. Teams that couple properly and add a learning drain twin and routing will look years ahead of teams that overlay rain on a DEM.
Judge-ability	**8/10**	MoES judges are likely to be domain scientists (IMD/NCCR/IITM). They will reward physical correctness, honesty about uncertainty and knowledge of Indian data. They will punish hand-waving. This is a feature for a serious team.
Data availability	**6/10**	Raw Doppler radar volumes and municipal drain GIS are not openly downloadable. Radar imagery is public; DEMs, roads, buildings and rain-gauge feeds are open. The blueprint's data playbook (Section 7) neutralises this risk with decoding pipelines, synthetic drain-graph inference and a replay engine.
36-hour feasibility	**7/10**	Only feasible with 8 weeks of preparation (pre-trained surrogate, pre-conditioned DEM, containerised services). With that preparation, the finale becomes integration and polish, not research.
Overall	**8.4/10**	**Choose it** if your team has at least two strong Python/geospatial/ML engineers, one systems engineer and one strong frontend/GIS engineer. Do not choose it if the team wants a CRUD app.

1.2 What the sponsor actually asked for (read the PS like a tender)

The PS text is unusually specific. Treat each clause as a mandatory deliverable and name your modules after them so the jury can tick boxes:

PS clause	What it means operationally	VARUNA module
"Fuse real-time rainfall nowcasts with high-resolution DEM and a graph-based model of the drainage network"	A coupled 1D (pipe network) – 2D (surface) hydrodynamic model driven by radar nowcasts. Not a heatmap.	Sky + Twin

"Takes high-resolution rainfall nowcasts (from Doppler Weather Radars) and instantly routes that volume across a 2D surface terrain model"	Radar reflectivity → rain rate → runoff → shallow-water routing on a DEM grid, fast enough for a 5-minute cycle.	Sky → Twin
"Directed graph (nodes as manholes/inlets, edges as pipes/canals)… calculate hydraulic capacity… predict where blockages or overcapacity will cause backflow onto the streets"	Manning capacity per edge, dynamic-wave routing, surcharge detection, and — crucially — a model of blockage, which is unknown. This is where VARUNA-Pulse becomes the differentiator.	Drain graph + Pulse
"Dynamic web-based GIS dashboard… street-by-street… water depth in centimetres… 0–3 hour window"	Vector tiles per road segment, time slider, probability layers, alerts.	Command
"API utility that can interface with navigation maps to suggest flood-safe alternative routes for emergency services, public transit and commuters"	Time-dependent, vehicle-profile-aware routing with a public API and a navigation-provider feed.	Route

1.3 Why most teams will lose this PS — and how you win

> **The threshold-map trap.** The typical submission will: fetch a rainfall forecast, multiply by a "runoff coefficient", pour it over a 30 m DEM using a fill-sink algorithm, colour the low points blue, and call it a nowcast. It ignores the drains entirely (or draws them as decoration), has no notion of time, no uncertainty, and cannot answer "which street floods at 18:30". MoES judges have seen this a hundred times.

You win by doing four things the trap cannot:

1. **Couple for real.** Water enters inlets at a computable rate, travels through pipes with finite capacity, backs up when the outfall is tide-locked, and surcharges out of manholes back onto the street. Show the surcharge happening on the map with the hydraulic grade line rising above the ground.
2. **Learn what you cannot see.** The drain network is invisible and clogged in unknown places. Instead of pretending you have the GIS, make every flood a measurement of the drain: traffic slowdowns, citizen photos, CCTV and sensors are assimilated to infer where blockages are. The twin gets smarter every monsoon.
3. **Make physics run at the speed of decisions.** Compile thousands of physics simulations into a graph neural surrogate so a 3-hour street-level forecast runs in well under a second — which is what makes 50-member probabilistic ensembles and interactive "what-if" possible in a control room.
4. **Turn prediction into action.** A flood map is not a product. An ambulance route that avoids the underpass that will be 60 cm deep at 18:40, a hospital reachability clock, a CAP alert to a ward officer and a pump-dispatch recommendation are products.

2. The problem, physically

2.1 The resolution gap

Operational NWP models resolve rainfall at 3–12 km grid spacing and update every few hours. Urban flooding is decided at 5–30 m: a 40 cm dip under a railway bridge, a kerb that channels water into an underpass, a nullah outfall that is below high-tide level for six hours a day. Even a perfect rainfall forecast at 12 km tells a control room nothing about whether the Andheri subway or the Minto Bridge underpass will be passable. The PS is right: knowing how much rain falls does not translate into knowing where streets flood. The translation needs terrain, surface roughness and the drainage network, in time.

2.2 Four mechanisms, one street

Mechanism	Physics	Indian examples
Pluvial (drain overload)	Rain intensity exceeds inlet capture and pipe capacity; water ponds in micro-depressions. Mumbai's legacy drains were designed for ~25 mm/h; BRIMSTOWAD upgrades target ~50 mm/h, while monsoon bursts routinely exceed 80–100 mm/h.	Hindmata, Sion Circle, Milan and Andheri subways (Mumbai); Minto Bridge, ITO (Delhi); Velachery, Pallikaranai edge (Chennai)
Fluvial (nullah/river overflow)	Urban streams (Mithi, Dahisar, Oshiwara; Cooum, Adyar; Barapullah, Najafgarh) exceed bank-full and spill; drains that discharge into them back up.	Mithi 2005; Adyar 2015 (Chembarambakkam release); Yamuna record 208.66 m in July 2023
Tidal / downstream lock	Gravity outfalls cannot discharge when sea or river stage is above the outfall invert; the whole network's boundary head rises and every upstream node surcharges earlier.	Mumbai high tide coinciding with cloudbursts; Chennai's Buckingham Canal during cyclones
Invisible / blocked drainage	Silt, plastic, construction debris and encroachment reduce effective conveyance by unknown fractions; municipal GIS is incomplete or paper-based. The network's true capacity is a latent variable.	Every city — the reason chronic waterlogging spots reappear each year despite desilting

A credible system must handle all four. Most "flood prediction" projects handle only the first, and only statically.

2.3 Why 0–3 hours is the right window

Below one hour, radar extrapolation of rain cells is skilful and a control room can still act: close an underpass, pre-position pumps, divert buses, hold trains at safe stations, route ambulances. Beyond three hours, convective cells decay and regenerate unpredictably and NWP takes over (this is IFLOWS' territory). The 0–3 h window is where the physical signal is strongest and the decisions are most concrete. It is also the window where the drain network matters most, because the time of concentration of an urban sub-catchment is 15–60 minutes: the network is the clock.

2.4 The cost of not knowing

- Mumbai, 26 July 2005: ~944 mm in 24 h at Santacruz; over a thousand lives lost; economic losses estimated in the billions of dollars. Hundreds of millimetres in a day have recurred in 2017, 2019, 2021, 2024 and 2025, each time stopping suburban trains and flooding underpasses.
- Chennai, December 2015 and Cyclone Michaung (December 2023): city-wide inundation, hundreds of deaths in 2015, multi-billion-dollar losses, airports and IT corridors closed for days.
- Delhi, July 2023 and June 2024: record Yamuna stage, record June rainfall, an airport canopy collapse, underpass drownings.

- Bengaluru (September 2022) and Hyderabad (October 2020): tech corridors and lake-bed encroachments flooded; multi-day disruption.

The recurring pattern in every post-mortem: authorities knew it would rain heavily, but not which streets would become impassable, or when. That sentence is the pitch.

3. Landscape and the gap

Judges from MoES built some of these systems. Know them, respect them, and position VARUNA as the missing street-scale layer that consumes their products rather than as a competitor.

System	Lead time	Resolution	Drain physics	Learns blockages	Probabilistic	Routing API	Notes
IFLOWS-Mumbai (MoES + BMC, 2020)	6–72 h	Ward / area	Partial (hydrologic-hydraulic)	No	Limited	No	Seven modules from data assimilation to DSS; NWP (NCMRWF/ IMD) + 165 rain gauges; tide and storm surge included. Excellent medium-range layer; not a street-level nowcast.
C-FLOWS Chennai (NCCR/ MoES, 2019)	Days	Zone / scenario maps	Scenario-based	No	Scenario	No	Pre-computed inundation scenarios for rainfall classes; decision support for GCC.
IIT Bombay Mumbai Flood app (2024)	Hourly, 24 h / 3-day	Neighbourhood rain; point water levels	No	No	No	No	60+ AWS, hyperlocal rain forecasts, water-level sensors at Mithi/Vakola, crowdsourced waterlogging reports. A great observation layer — and a potential data partner.
IMD nowcasts	0–3 h	District / city	No	No	No	No	Thunderstorm/ heavy-rain nowcasts from radar; the meteorological input VARUNA consumes.
Google Flood Hub	Up to 7 days	River reaches	No	No	Yes	No	Riverine AI forecasts; not urban pluvial, not street-level.

Global commercial (Previsico UK, FloodMapp AU, 7Analytics NO, Fathom UK)	0–24 h	Street / property	Some	No	Some	Some	Proof that street-level pluvial nowcasting is a funded, paying market abroad (insurers, utilities, councils). None operate at Indian monsoon intensities or with Indian data.
Academic GPU solvers (SynxFlow/ HiPIMS, LISFLOOD-FP, ItzÏ)	n/a	2–10 m	Some (SWMM coupling)	No	No	No	Open-source building blocks. A 2026 npj Natural Hazards paper on GPU city-scale forecasting notes that drainage data are rarely accessible — the exact gap VARUNA-Pulse closes.
VARUNA	**0–3 h, 5-min cycle**	**Road segment; 5–30 m cells**	**Full 1D–2D coupled**	**Yes (assimilation)**	**Yes (ensembles)**	**Yes (time-dependent, vehicle-aware)**	Consumes IMD/NCMRWF/ IFLOWS products as inputs and boundary conditions.

> **Positioning sentence for the jury:** "IFLOWS tells Mumbai a day ahead which wards are at risk. VARUNA tells the ward officer three hours ahead which junction will be 45 cm deep, which drain is choking, where to send the pumps, and which road the ambulance should take — and it learns the drain network from every flood it sees."

4. The big idea: VARUNA

VARUNA (the Vedic deity of waters; expanded as *Vector-graph Assimilated Runoff & Urban Nowcasting Architecture*) is a self-correcting digital twin of a city's water: sky, surface and sewer, coupled, probabilistic, and fast enough to sit in a 5-minute operational loop. It is built as six engines that share one graph.

Engine	What it does	The "why didn't we think of that" moment
VARUNA-Sky	Turns Doppler radar reflectivity, rain gauges, satellite and NWP into a calibrated, probabilistic 0–3 h rainfall ensemble at 500 m–1 km, every 5 minutes.	Gauge-adjusted radar with an adaptive Z–R relation per storm, plus stochastic ensembles instead of one deterministic rain field. Uncertainty is carried, not hidden.
VARUNA-Twin	GPU-accelerated 2D shallow-water surface routing on a hydro-conditioned DEM, coupled to a 1D dynamic-wave drainage graph with tidal/river boundaries.	Full-city deterministic 3-hour physics run in about a minute on one consumer GPU; nested 5 m grids at chronic hotspots.
VARUNA-Pulse	The living drain. Assimilates traffic-speed anomalies, citizen photo reports, CCTV water segmentation, IoT level sensors and SAR to infer per-pipe blockage factors and inlet clogging with an ensemble Kalman filter.	**Everyone tries to map the invisible drain. VARUNA makes the city reveal it: every flood is a measurement.** Traffic jams become flood sensors; the twin's drain map gets more accurate each monsoon and outputs a desilting priority list worth crores.
VARUNA-Flash	A graph neural surrogate trained on thousands of Twin simulations; emulates the coupled physics in under a second per member.	Physics compiled into a neural network. Enables 50-member probabilistic street forecasts and interactive what-if ("clean these 14 drains", "+30% rain") in a control room, on a laptop.
VARUNA-Route	Time-dependent, vehicle-profile-aware routing and reachability isochrones; feeds navigation providers, transit (GTFS-RT) and emergency dispatch.	Roads are not "closed" or "open": they have a depth curve and a "safe-until" time per vehicle class. Hospitals get a reachability clock.
VARUNA-Command	GIS operations console, public map, CAP/WhatsApp/SMS alerts, and a pump-dispatch optimiser that assigns dewatering pumps to hotspots ahead of the peak.	Prediction converted to a dispatch order. The ward officer receives "Hindmata > 45 cm at 18:20, move pumps P-12 and P-15 now", not a colour map.

4.1 The seven unfair advantages (your moat)

1. **One shared heterogeneous graph.** Surface cells, road segments, inlets, manholes, pipes, outfalls and gauges live in the same graph. Physics, assimilation, surrogate and routing all operate on it. Nobody has to build glue.
2. **Learning drain capacity from human-sensed signals.** Published research on urban flood nowcasting with heterogeneous community features shows the value of fusing physics with human-sensed data; VARUNA is the first to close the loop into the hydraulic parameters themselves.
3. **Physics-to-neural compilation.** The surrogate makes probabilistic, interactive nowcasting affordable; the physics twin keeps it honest and keeps generating fresh training data every cycle.
4. **City-in-a-box onboarding.** A scripted pipeline builds a city from open data (DEM, OSM roads/drains, building footprints, land cover, rain gauges) and a synthetic drain graph in days, then refines as municipal GIS arrives. This is the scaling story for 18 UFRMP cities and beyond.
5. **Replay engine.** Any historical event can be streamed as if live. Judges watch VARUNA "predict" 2 July 2019 Mumbai two hours before it happened, with the real tweets and photos appearing on the map as ground truth.

6. **Standards-native.** OGC API Tiles/Features, STAC for rasters, CAP for alerts (the protocol NDMA's Sachet uses), GTFS-RT for transit, SWMM .inp import for municipal models. Governments can adopt without lock-in.
7. **Honest uncertainty.** Calibrated probabilities, reliability diagrams and explicit "confidence collapse after 90 minutes" messaging. Scientists trust systems that know what they do not know.

4.2 The one-line story

> **"VARUNA is a digital twin of the city's water that learns its hidden drains from every flood, runs its physics in milliseconds, and tells emergency services which street will be impassable, when, and how to get around it — three hours early."**

5. System architecture and the 5-minute cycle

VARUNA is an event-driven pipeline that runs a full cycle every five minutes (radar cadence is 10 minutes; gauges 15; traffic 1–2; citizen reports continuous). Static layers (DEM, roads, buildings, drain graph, roughness) are pre-processed once per city and versioned. Everything dynamic flows through a message bus into stateless workers, and every cycle writes a versioned run so any forecast can be replayed and audited.

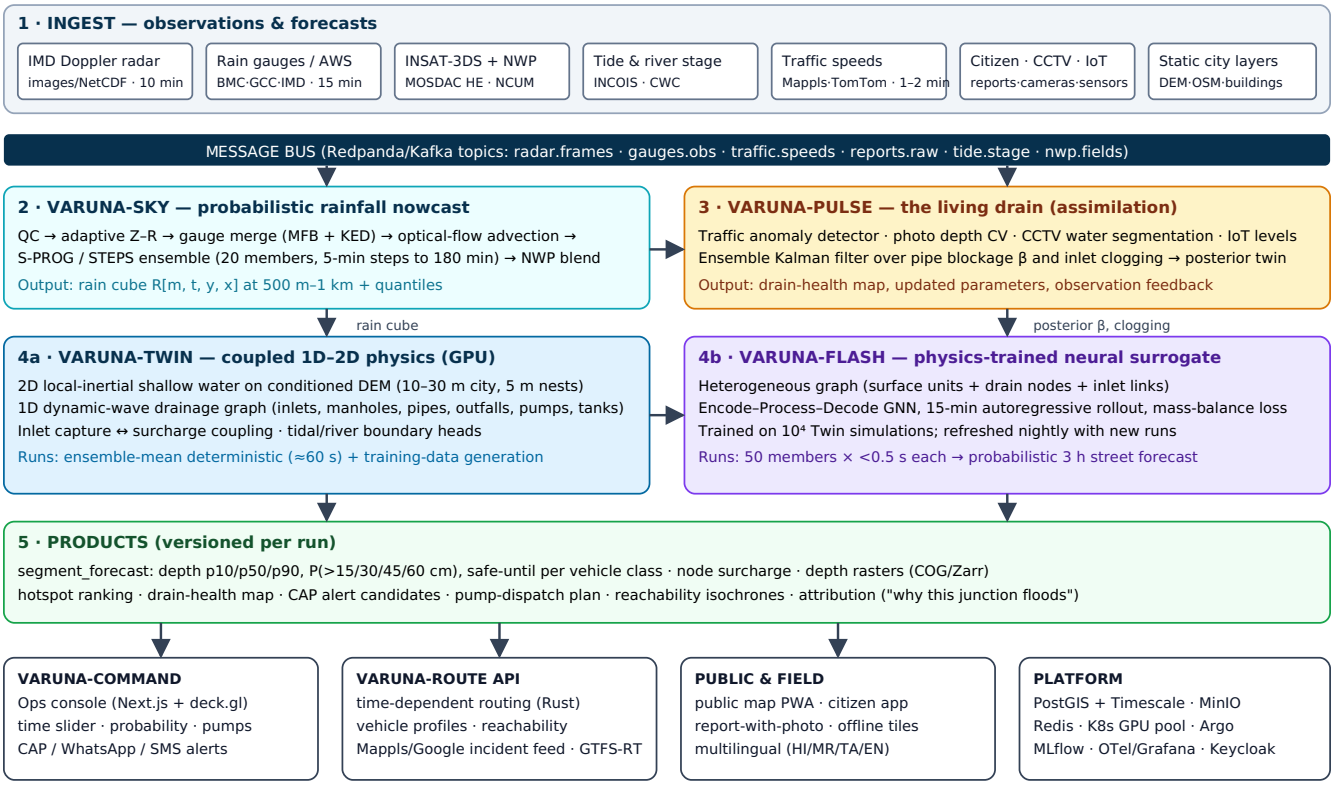
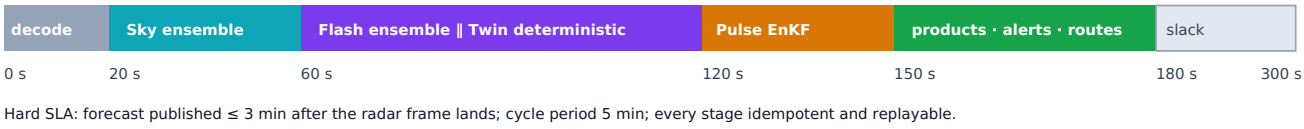


Figure 1 — VARUNA logical architecture. One shared city graph; five layers; every arrow is a versioned message (Twin → Flash arrow = training runs).

5.1 The 5-minute cycle: timing budget



Hard SLA: forecast published ≤ 3 min after the radar frame lands; cycle period 5 min; every stage idempotent and replayable.

Figure 2 — Wall-clock budget for one operational cycle on a single GPU node.

Stage	Budget	What happens	Compute
Decode & QC	0–20 s	Radar image/NetCDF → georeferenced dBZ grid; clutter and attenuation masks; gauge and tide values fetched; traffic snapshot pulled.	CPU, Python
Sky	20–60 s	Adaptive Z–R, gauge merging, optical flow, 20-member STEPS ensemble to +180 min, NWP blend, quantiles.	CPU (pySTEPS), optional GPU

Flash + Twin	60–120 s	Surrogate runs 50 members (rain members × parameter draws) in seconds; the physics Twin runs the ensemble-mean deterministically for consistency, hotspot nests and training data.	GPU (PyTorch + CUDA kernels)
Pulse	120–150 s	Assimilate observations that arrived since last cycle; update β posterior; flag disagreement between model and observations.	CPU/GPU (surrogate-driven EnKF)
Products	150–180 s	Segment table, rasters, vector tiles, hotspot ranking, alert candidates (with hysteresis), pump plan re-solve, routing overlay rebuild, webhooks.	CPU, PostGIS, Rust route service

5.2 Design principles

- **Everything is a versioned run.** Run ID = city + cycle timestamp + model versions. Every product carries its run ID so an ambulance route can be traced to the exact rain field and drain parameters that produced it.
- **Stateless workers, stateful stores.** Workers scale horizontally; state lives in PostGIS/Timescale, Redis (hot), and object storage (rasters, ensembles as Zarr).
- **Graceful degradation.** No radar? Sky falls back to gauge + satellite + NWP with wider uncertainty. No GPU? Flash runs on CPU (slower, fewer members) and Twin runs at 30 m only. No traffic API? Pulse uses reports and sensors. The dashboard shows which mode it is in.
- **Replay-first.** The same code path serves live feeds and historical replays. This is what makes the demo bulletproof and validation continuous.
- **Open interfaces.** SWMM .inp import for municipal drain models, OGC/STAC/CAP/GTFS-RT out. A city can leave and keep its data.

6. Science & mathematics of every engine

This section is the reference your engineers implement from. Each subsection states inputs, equations, algorithm, outputs, known failure modes and the V1/V10/V100 boundary.

6.1 VARUNA-Sky: probabilistic rainfall nowcasting

Inputs

- **Doppler Weather Radar.** IMD operates S-band and C/X-band radars covering Mumbai, Delhi (Palam, Lodhi Road, Ayanagar), Chennai and other metros; reflectivity products (PPI, MAX-Z, CAPPI) are published as images every ~10 minutes on mausam.imd.gov.in with 3-hour animations. Raw NetCDF/ HDF volumes are obtained from IMD/MoES on request — as the PS sponsor, MoES is the natural provider of benchmark event data. V1 decodes the public images (colour-scale → dBZ, georeferenced by the radar site and range rings) and ingests NetCDF when provided; V10 uses a direct MoES feed.
- **Rain gauges / AWS.** BMC's automatic weather stations (60+ used by IIT-B, 165 in IFLOWS), Greater Chennai Corporation gauges, IMD observatories (Santacruz, Colaba, Safdarjung, Nungambakkam). Used for radar bias correction and verification.
- **Satellite.** INSAT-3DS Hydro-Estimator rainfall (MOSDAC), ~15 min, ~4 km — fallback when radar is down or beam-blocked; GPM IMERG for training archives (too latent for live use).
- **NWP.** IMD-GFS, NCMRWF NCUM-R (~4 km), or the WRF runs that feed IFLOWS — used for the 1–3 h blend and for the "beyond 3 h" context.

Quantitative precipitation estimation (QPE)

$Z = a \cdot R^b \;\Rightarrow\; R = (Z / a)^{1/b}$ [Z in $mm^6\ m^{-3}$ (from dBZ: $Z = 10^{dBZ/10}$), R in $mm\ h^{-1}$]
Marshall–Palmer: a = 200, b = 1.6 · Tropical convective (Rosenfeld): a = 250, b = 1.2 · **VARUNA adaptive:** fit (a, b) each hour by least squares on log R_{gauge} vs log Z over co-located gauge–radar pairs of the last 60 min; clamp to physical bounds.

After Z–R conversion, apply a **mean-field bias** factor MFB = ΣG_i / ΣR_i (gauges G, radar R) and a spatially varying residual via **Kriging with External Drift** (radar field as drift) so the merged product honours gauges exactly and radar structure everywhere else. Quality control removes ground clutter (velocity ≈ 0 with high texture), anomalous propagation, and applies attenuation correction for C/X-band. Beam-blockage maps are computed once from the DEM.

Extrapolation nowcast (pySTEPS)

1. Motion field: dense Lucas–Kanade optical flow on the last three radar frames (10-min spacing), smoothed; alternative: DARTS or a learned flow.
2. Deterministic member: **S-PROG** — decompose the field into spatial scales (FFT cascade), advect each with semi-Lagrangian backward integration, apply per-scale AR(2) autoregression so small scales decay faster than large ones (this is the physics of convective predictability).
3. Stochastic ensemble: **STEPS** — add spatially correlated noise to the cascade and perturb the motion field to generate M = 20 members with realistic growth/decay; steps of 5 min out to 180 min.
4. Probability matching to preserve the observed rain-rate distribution; quantiles p10/p50/p90 per pixel-time.

Blending with NWP for 1–3 h

$R_{blend}(\tau) = w(\tau) \cdot R_{ext}(\tau) + (1 - w(\tau)) \cdot R_{NWP,bc}(\tau), \quad w(\tau) = \exp(-\tau / \tau_s)$, $\tau_s \approx 60$–90 min (learned per city/season from verification); $R_{NWP,bc}$ is quantile-mapped against gauges.

Deep-learning upgrade V10

Train a NowcastNet/Earthformer-style spatio-temporal model (or a DGMR-like conditional generator) on Indian radar archives obtained through MoES, with INSAT and gauge channels; loss = weighted CSI at 20/40 mm h^{-1} + spectral loss to avoid blurring. Use it as an additional ensemble member family, not a replacement — ensembles of different methods are more skilful than any single method.

Outputs and failure modes

Output: rain cube R[m, t, y, x] (M members × 36 steps × grid at 500 m–1 km) plus exceedance probabilities. Known failure modes: rapid convective initiation not present in the last frames (unavoidable — the reason for probabilistic output and NWP blend), radar attenuation in extreme rain (flag and lean on gauges/satellite), and Z–R drift across storm types (adaptive fit).

6.2 Surface hydrology and terrain conditioning

Effective rainfall

$R_{eff} = I \cdot \max(0, R - d_s) + (1 - I) \cdot \max(0, R - f)$ I = impervious fraction of the cell, d_s = depression storage (1–2 mm), f = infiltration rate.

Green–Ampt: $f(t) = K_s\,[1 + \psi\,\Delta\theta / F(t)]$ · or SCS-CN: $Q = (P - 0.2\,S)^2 / (P + 0.8\,S)$, $S = 25400/CN - 254$ (mm), CN ≈ 90–98 for saturated monsoon urban soils.

Imperviousness comes from ESA WorldCover (10 m) refined by building footprints (Google Open Buildings v3 and Microsoft footprints cover Indian cities) and OSM road polygons. Curve numbers come from soil class (NBSS&LUP) × land use.

DEM: the hard truth and the plan

> **The single biggest physical limitation is vertical DEM accuracy.** Open DEMs (Copernicus GLO-30, FABDEM, ALOS AW3D30 at 30 m; CartoDEM at 10 m / 2.5 m for some cities) carry vertical errors of the order of metres; street flooding is decided by decimetres. Say this to the jury before they say it to you. Then explain the mitigation: (i) use the DEM for *flow direction, connectivity and ponding pattern*, which it captures well; (ii) calibrate absolute depths at chronic spots against observations (sensors, photos, historical marks); (iii) budget drone/aerial LiDAR (≤ 15 cm vertical accuracy) as a pilot deliverable — UFRMP funds exactly this kind of data acquisition; (iv) apply ML super-resolution of the DEM using building footprints and road priors as a research upgrade.

Hydro-conditioning pipeline (WhiteboxTools / RichDEM / GDAL)

1. Reproject to a metric CRS (UTM 43N Mumbai/Delhi, 44N Chennai); resample to 10 m (city) and 5 m (nests).
2. Burn buildings as raised blocks (+5 m or footprint height) so water flows around them; treat building cells as blocked or very high roughness.
3. Carve road centrelines (−0.1 to −0.2 m relative to footpaths) where LiDAR is absent, to reproduce kerb channelisation; preserve known underpasses and subways as true sinks (do not fill them).
4. Breach culverts and bridges using OSM tags (bridge=yes, tunnel=culvert, waterway=drain) so water is not blocked by road embankments.
5. Remove only spurious pits (area < threshold) with a priority-flood breaching algorithm; every remaining depression becomes a candidate hotspot (this list alone impresses municipal engineers when it matches their chronic-spot registers).
6. Roughness map (Manning's n): asphalt 0.013–0.02, open ground 0.03–0.05, vegetation 0.05–0.1, buildings 0.3+ (or blocked), water 0.03.

6.3 VARUNA-Twin: GPU 2D surface routing

The surface solver uses the **local inertial approximation** of the 2D shallow-water equations (the LISFLOOD-FP formulation of Bates et al., 2010), which keeps the local acceleration term, drops advection, and is stable, mass-conservative and embarrassingly parallel — ideal for GPUs and for shallow urban flows. Full Godunov-type solvers (as in HiPIMS/SynxFlow) are reserved for hotspot nests where supercritical flow through underpasses matters.

Flux between cells (per unit width), for each face:
$q^{t+\Delta t} = [\, q^{t} - g \cdot h_f \cdot \Delta t \cdot \partial(h + z)/\partial x \,] \,/\, [\, 1 + g \cdot \Delta t \cdot n^2 \cdot |q^{t}| \,/\, h_f^{10/3} \,]$
h_f = max(h + z at both cells) − max(z at both cells) (flow depth at the face)
Continuity per cell: $h^{t+\Delta t} = h^{t} + \Delta t \cdot [\, \Sigma q_{in} - \Sigma q_{out} \,] / \Delta x + \Delta t \cdot (R_{eff} - Q_{inlet}/A + Q_{surcharge}/A)$
Stable time step (CFL): $\Delta t = \alpha \cdot \Delta x / \sqrt{(g \cdot h_{max})}$, $\alpha \approx 0.7$

Numbers that make it real-time

- Mumbai city ≈ 600 km²; at 10 m that is 6 million cells; at 30 m, 0.7 million. With $h_{max} \approx 1$ m, $\Delta t \approx 2$ s at 10 m → ~5,400 steps for a 3-hour horizon.
- A single stencil update is ~30 flops; a consumer GPU (RTX 4070 class) sustains 1–3 billion cell-updates per second in a well-written CUDA/Numba kernel → a full 10 m deterministic 3-hour Mumbai run in roughly 15–60 s. At 30 m it is a few seconds. This is why the Twin runs every cycle and can generate training data continuously.
- Nested 5 m grids at the top ~50 chronic hotspots (each ~1 km²) add negligible cost and give decimetre-scale detail where it matters.

Boundary conditions

Sea and creek boundaries take tide stage from INCOIS/Survey of India tide predictions (plus storm-surge adjustments when IMD issues them); river boundaries take CWC gauge stage or barrage releases (Chembarambakkam, Hathnikund/Yamuna). Internal boundaries: pumping stations (Mumbai has several coastal stormwater pumping stations) and holding tanks (e.g., the Hindmata underground tanks) are modelled as controlled sinks/sources.

Implementation notes

V1: Numba-CUDA or CuPy kernels in Python (fast to write). **V10**: Rust + CUDA (via `cust`) or Rust + wgpu compute shaders, exposed to Python through PyO3; the same Rust core compiles to WebAssembly for in-browser local what-if simulations. Mass balance is asserted every 100 steps ($|\Delta V_{stored} - V_{in} + V_{out}| < 0.1\%$).

6.4 The drainage graph: 1D hydraulics, capacity, surcharge

Graph definition

Element	Attributes
Node (inlet / manhole / junction)	id, geometry, ground elevation z_g, invert elevation z_i, storage area A_s, inlet type (kerb/grate/combination), inlet length L_i and opening area A_o, clogging factor $\kappa \in [0,1]$ (mean, variance), catchment cells served
Node (outfall)	boundary type (free, fixed stage, tidal series), flap gate flag, receiving water body
Node (storage / pump / gate)	storage curve, pump curve Q(H), on/off rules, gate opening
Edge (conduit / canal / nullah)	from, to, length L, shape (circular/box/trapezoid), diameter or width×height, Manning n, slope S, blockage factor $\beta \in [0,1]$ (mean, variance), last-desilted date, confidence (surveyed / inferred)

Capacity and routing

Full-flow capacity (Manning): $Q_{full} = (1/n) \cdot A \cdot R_h^{2/3} \cdot S^{1/2}$, with effective area $A_{eff} = (1 - \beta) \cdot A$ and R_h recomputed for the blocked section.
Dynamic wave (Saint-Venant 1D) per conduit: $\partial A/\partial t + \partial Q/\partial x = 0$; $\partial Q/\partial t + \partial(Q^2/A)/\partial x + g\,A\,\partial H/\partial x + g\,A\,S_f = 0$, $S_f = n^2\,Q\,|Q| / (A^2\,R_h^{4/3})$
Node continuity: $dH_j/dt = (\Sigma Q_{in,j} + Q_{inlet,j} - \Sigma Q_{out,j} - Q_{surch,j}) / A_{s,j}$
Surcharge condition: $H_j > z_{g,j}$ (hydraulic grade line above ground) $\Rightarrow Q_{surch,j} > 0$ (§6.5). Pressurised flow is handled with the Preissmann slot or SWMM's surcharge algorithm.

V1: EPA SWMM5 through **PySWMM** (proven dynamic-wave engine; the municipal BRIMSTOWAD and C-FLOWS models can be exported to SWMM/.inp), stepped in lock-step with the 2D solver. **V10**: a Rust dynamic-wave solver on the same graph for GPU-side batching (needed for thousands of training runs). The **backflow** the PS asks about is exactly the case where downstream head (tide-locked outfall, surcharged trunk) exceeds upstream head so Q reverses sign in the momentum equation — the solver handles it natively and the dashboard renders reversed-flow edges in red.

6.5 1D–2D coupling: how water enters and escapes the drains

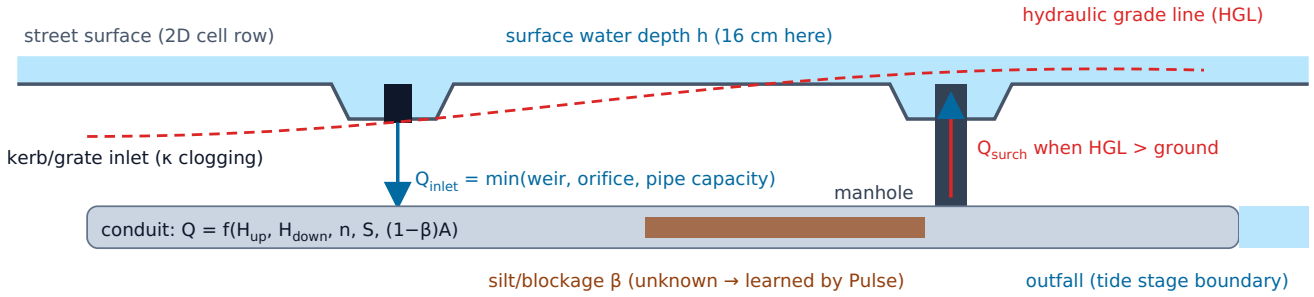


Figure 3 — Bidirectional 1D–2D exchange at a street: capture at inlets, backflow/surcharge at manholes when the HGL rises above ground.

Surface → drain (inlet capture), depth h on the inlet cell:
$Q_{weir} = C_w \cdot L_i \cdot h^{3/2}$ ($C_w \approx 1.66$ SI) · $Q_{orifice} = C_o \cdot A_o \cdot \sqrt{(2\,g\,h)}$ ($C_o \approx 0.6$)
$Q_{inlet} = (1 - \kappa) \cdot \min(Q_{weir}, Q_{orifice}, Q_{avail})$, Q_{avail} = max(0, capacity remaining at the node given H_j)
Drain → surface (surcharge), when $H_j > z_{g,j} + h$:
$Q_{surch} = C_o \cdot A_m \cdot \sqrt{(2\,g\,(H_j - z_{g,j} - h))}$ (A_m = manhole opening area; reversed when the surface is higher, i.e., the manhole drains the street)

Synchronisation: the 2D solver advances with its CFL step; the 1D solver advances with its own internal sub-steps; exchange fluxes are frozen over a sync interval Δt_{sync} = 5 s and limited so a cell cannot be drained below zero (flux limiter). This is the same coupling pattern used by ItzÏ (GRASS + SWMM) and commercial 1D–2D packages; VARUNA's contribution is the clogging factor κ and blockage β being *random variables* with posteriors, not constants.

6.6 VARUNA-Pulse: learning the invisible drain (data assimilation)

The PS says the network is "invisible" and asks to predict where blockages cause backflow. The honest answer is that nobody knows the blockages — so VARUNA estimates them. The state to be estimated is θ =

{β_e for pipes, κ_j for inlets, plus a few global corrections (rain bias, roughness)}. Observations y arrive from five families:

Observation	How it becomes a measurement	Level
Traffic as sensor	For each road segment, compare live speed v(t) (Mappls/TomTom/Google traffic feeds) with the segment's historical profile for the same weekday/time μ(t), σ(t). Compute z = (v − μ)/σ. During rain, a persistent z < −2.5 not explained by network-wide congestion (control for neighbouring dry segments) is a "probable flooding" observation with a depth prior (speed collapse to <5 km/h ≈ depth > 20 cm). Latency 1–2 min, coverage city-wide, zero hardware.	V1
Citizen photo reports	App/WhatsApp report with photo; a segmentation model finds the water line against reference objects of known height (car wheels ≈ 0.6–0.7 m diameter, kerbs 15 cm, tyres, bus steps) to estimate depth ±10 cm. Reports are geolocated and de-duplicated; trust score per reporter.	V1 ingestion; CV depth V10
CCTV	Municipal/traffic-police camera frames (Mumbai has thousands) through a water-on-road segmentation model; output: wet/flooded/impassable class + depth estimate at known camera geometries.	V10
IoT level sensors	Ultrasonic/pressure sensors at chronic spots and in manholes (IIT-B already operates some at Mithi/Vakola; UFRMP funds more). Direct depth/HGL observations — the strongest constraint on β.	V10
Satellite SAR	Sentinel-1 flood extents after large events for whole-city validation and seasonal re-calibration.	V10

The update

Ensemble Kalman filter over parameters θ ($N_e = 50$ members drawn from the current posterior):

forecast: $y_k^f = H(\theta_k)$ (H = Flash surrogate mapping parameters + rain to depth at observation locations)

gain: $K = P_{\theta y}\,(P_{yy} + R)^{-1}$, update: $\theta_k^a = \theta_k^f + K\,(y + \varepsilon_k - y_k^f)$, $\varepsilon_k \sim N(0, R)$

θ constrained to [0,1] via logit transform; localisation limits each observation's influence to pipes within its hydraulic neighbourhood (upstream/downstream 2–3 hops).

Priors: β ~ Beta(a, b) with means from pipe age, months since last desilting, waste-complaint density (municipal helpline data) and land use (markets and informal settlements clog faster). Between events, the posterior persists and decays slowly toward the prior (desilting resets it). **Products:** a *drain-health map* (posterior mean and uncertainty per pipe) that directly ranks pre-monsoon desilting — a budget-relevant output for corporations that spend hundreds of crores a year on desilting — and a *model-observation disagreement* layer that tells engineers where the twin is wrong.

> **Why this is the idea judges will envy:** it converts the PS's hardest data problem (no drain GIS) into a learning problem with free, city-wide sensors, and it produces a second product (drain health) with its own budget line. And it is scientifically sound: parameter estimation with EnKF using surrogate models is standard in hydrology; applying it to blockage factors with human-sensed observations is the novel synthesis.

6.7 VARUNA-Flash: the physics-trained neural surrogate

Graph construction

Aggregate the 2D grid into surface units — one per road segment plus DEM-derived micro-catchments (hydrologic response units of ~0.5–2 ha), giving ~20–60k surface nodes per city; add the drain nodes (~10–50k). Edges: surface adjacency (with flow-direction weights), conduits, inlet links (surface unit → inlet node). Node features: elevation stats (min, mean, depression depth), imperviousness, roughness, area,

current depth; drain features: z_g, z_i, capacity, β, κ; edge features: length, slope, capacity, direction. Forcing: rainfall per surface unit for the next 12 steps of 15 min.

Model

- Encode–Process–Decode architecture in the MeshGraphNets/GraphCast family: MLP encoders → 12–15 rounds of edge-conditioned message passing (hidden size 128, residual, LayerNorm) → decoders for surface depth Δh and node head ΔH.
- Autoregressive rollout at 15-min steps to 3 h with rainfall forcing; noise injection during training for rollout stability.
- Loss: MSE on log(1 + h) and H, an exceedance-weighted term (so 30–60 cm events are not averaged away), and a soft mass-balance penalty per catchment (Σ storage change = rain − outflow − infiltration).
- Uncertainty: run the surrogate over the Sky rain members and the Pulse parameter members (50 combinations) → probabilities; MC-dropout adds structural uncertainty.
- Explainability: gradients of hotspot depth with respect to β_e rank the pipes "responsible" for a junction — the dashboard's "why does this flood" panel.

Training data

Generate 5,000–20,000 Twin runs pre-hackathon: IMD IDF-based design storms (2–100-year return periods, Chicago/alternating-block hyetographs and moving-cell storms), historical radar events and their STEPS perturbations, tide levels sampled from the tidal range, and β/κ sampled from the priors. At ~30–60 s per run on one GPU this is 2–7 GPU-days — feasible on a rented cloud GPU or a lab workstation. Each operational cycle adds a fresh run; nightly fine-tuning keeps the surrogate current. Report surrogate-vs-physics RMSE (target ≤ 5 cm on depth, ≤ 10 cm at hotspots) and CSI at 30 cm (target ≥ 0.85) in the pitch.

V1: a compact surrogate for the demo city (or, if training slips, a CPU-fast 30 m physics run with 5 members — still probabilistic). **V10**: full-city surrogate refreshed nightly. **V100**: federated training across cities so a new city starts with transferred hydraulic knowledge.

6.8 Street products: depth, probability, impassability

Product	Definition
Segment depth series	For each road segment (OSM way split at intersections): depth quantiles p10/p50/p90 at 15-min steps to +180 min; maximum depth and time-to-peak.
Exceedance probabilities	P(h > 15 cm), P(h > 30 cm), P(h > 45 cm), P(h > 60 cm) per step.
Impassability by vehicle class	Two-wheeler ≥ 15 cm; car ≥ 30 cm (water can float a car at ~30 cm, per "Turn Around, Don't Drown" guidance); bus/truck/SUV ≥ 45 cm; specialised rescue vehicles ≥ 60 cm; pedestrians unsafe when h·v ≥ 0.5 m^2/s or h ≥ 30 cm. Thresholds are configurable per fleet.
Safe-until time	First time at which P(h > $threshold_{vehicle}$) exceeds the operator's risk tolerance (default 0.5) — shown to routing and to citizens as "passable until 18:25".
Hotspot ranking	Segments ranked by expected impact = P(impassable) × exposure (traffic volume, hospitals, transit lines, population within 300 m).
Node surcharge	Manholes/inlets predicted to overflow, with the responsible pipes (gradient attribution) and the drain-health posterior.
Reachability	Isochrones from hospitals, fire stations and disaster shelters under the forecast, per vehicle class, per time step; "reachability collapse" alerts when a critical facility's 15-min catchment shrinks below a threshold.

6.9 VARUNA-Route: flood-aware navigation and reachability

Road network from OSM (OSMnx) stored in PostGIS; the routing engine is a Rust service (Axum) holding the graph in memory. Each edge e has a base travel time t_e and a time-dependent penalty from the latest run:

$c_e(\tau, v) = t_e \cdot \varphi(h_e(\tau))$ if $P(h_e(\tau) > \theta_v) < p_{max}$, else ∞
$\varphi(h) = 1$ for $h < 5$ cm; linearly rising to 3× at θ_v (slow wading traffic); p_{max} = operator risk tolerance (0.5 default, 0.2 for ambulances)

- Algorithm: time-dependent Dijkstra/A* with the FIFO property (departure-time-dependent costs; arrival time at each node determines which 15-min depth slice applies). For city scale and sub-100 ms latency, rebuild profile-specific contraction hierarchies every cycle (5-min cadence permits it) — or use Valhalla/OSRM with dynamic edge penalties as the V1 shortcut.
- Vehicle profiles: ambulance, fire tender, bus, car, two-wheeler, pedestrian (with h·v hazard), plus custom fleet profiles (ground clearance, wading depth).
- Reachability: multi-source Dijkstra from critical facilities per time slice → isochrone polygons; "reachability collapse" events feed alerts.
- Integrations: navigation providers consume a **road-condition feed** (GeoJSON/Protobuf of impassable and degraded segments with validity windows) — Mappls and Google-style incident/closure feeds; transit agencies receive **GTFS-Realtime service alerts**; dispatch centres call the route API directly; the public map shows safe corridors.

6.10 VARUNA-Command: alerts and pump dispatch optimisation

Alert logic

Alerts are generated per segment/ward when P(h > threshold) crosses a trigger with **hysteresis** (trigger at 0.6, clear at 0.3) and **persistence** (two consecutive cycles) to avoid flapping. Messages follow the **Common Alerting Protocol (CAP)** so they can enter NDMA's Sachet / cell-broadcast pipeline, and are mirrored to WhatsApp (Business API), SMS and the dashboard. Escalation matrix: ward officer → disaster-management control room → police/traffic → transit operators → public.

Pump dispatch as an optimisation problem

Given hotspots i with forecast excess inflow volume $V_i(\tau)$ (from the Twin/Flash: inflow − drain outflow), pumps p with capacity c_p and travel time $d_{p,i}$, exposure weights w_i:
minimise $\Sigma_i w_i \cdot \Sigma_\tau \max(0, V_i(\tau) - \Sigma_p x_{p,i} \cdot c_p \cdot (\tau - d_{p,i})^+)$ s.t. $\Sigma_i x_{p,i} \le 1$, $x \in \{0,1\}$
Solved as a MILP (OR-Tools/SCIP) every 15 min; a greedy fallback guarantees an answer in < 1 s.

Output: a dispatch board ("move P-12 from Parel to Hindmata now; ETA 25 min; prevents ~40 min of >45 cm depth"). The same machinery schedules pumping-station operation and holding-tank drawdown ahead of the peak, and — V100 — model-predictive control of gates and reservoir releases.

6.11 Validation, metrics and benchmark events

Metric	Definition and target
CSI / POD / FAR (per segment, per threshold)	Contingency scores for "flooded > 30 cm within the 3-h window" against observed points (sensors, reports, news-verified). Targets: CSI ≥ 0.6 at chronic spots, FAR ≤ 0.3.
Depth error	MAE ≤ 10 cm at sensor points; bias reported separately.
Timing error	$\lvert t_{peak,pred} - t_{peak,obs} \rvert \le 30$ min at chronic spots.
Probabilistic skill	Brier score and reliability diagram for P(>30 cm); ROC AUC ≥ 0.85.

Rain nowcast skill	CSI at 20 and 40 mm/h by lead time versus gauges; the skill curve versus lead time is the honest picture judges want.
Latency	Frame-to-product ≤ 3 min; API p95 ≤ 200 ms.
Routing value	Share of emergency trips that would have hit an impassable segment under naïve routing versus VARUNA routing, on replayed events.

Benchmark events (build the replay library before the finale): Mumbai 26 July 2005 (reference case), 29 August 2017, 1–2 July 2019, 5 August 2020 (storm-surge coincidence), July 2024 overnight burst, May 2025 pre-monsoon flooding; Chennai 1–2 December 2015, November 2021, Cyclone Michaung 3–5 December 2023; Delhi 8–9 July 2023 and 28 June 2024; Bengaluru September 2022; Hyderabad October 2020. Ground truth from sensors (where they existed), municipal chronic-spot reports, geotagged social media and news photos with timestamps, and Sentinel-1 SAR extents for the largest events.

7. Data playbook for India

Data is the difference between a slide and a system. This table is your acquisition checklist; start on day one of preparation, because the slow items (radar volumes, drain GIS, traffic API keys) take weeks.

Layer	Primary source	Resolution / cadence	Access route	Fallback
Radar reflectivity	IMD DWR products (mausam.imd.gov.in radar pages, 3-h animations; raw volumes from IMD/MoES)	~1 km, 10 min	Public images now; write to the MoES SPOC named in the PS for benchmark NetCDF/HDF volumes; IMD data-supply portal for archives	INSAT-3DS HE (MOSDAC), gauge-only nowcast, global radar aggregators where they cover India
Rain gauges	BMC AWS (disaster-management portal), GCC gauges, IMD observatories; IIT-B Mumbai Flood platform	Point, 15 min	Public dashboards/ portals; request API/CSV from MCGM DM cell and IIT-B IDPCS	Scrape public tables; IMD daily/hourly
NWP	IMD-GFS, NCMRWF NCUM-R; ECMWF open data; GFS (NOAA)	4–25 km, 3–6 h	NCMRWF/IMD portals; NOAA NOMADS and ECMWF open data are freely downloadable	GFS only
Tide & surge	INCOIS tide predictions; Survey of India tide tables; IMD surge advisories	Point, 10 min–hourly	Public	Harmonic prediction from constituents
River stage / releases	CWC gauges (Yamuna), PWD Tamil Nadu (Chembarambakkam), Delhi barrage data	Point, hourly	Public dashboards / press	Modelled boundary
DEM	CartoDEM (NRSC Bhuvan; 10 m, 2.5 m in places), Copernicus GLO-30, FABDEM (bare-earth), ALOS AW3D30, NASADEM; TanDEM-X 12 m; city LiDAR where it exists	2.5–30 m	Bhuvan registration; Copernicus/ OpenTopography downloads; FABDEM (research licence)	Fuse several DEMs; ICESat-2 for vertical calibration; drone LiDAR in pilot
Buildings	Google Open Buildings v3, Microsoft Global ML Building Footprints, OSM	Polygon	Public downloads	OSM only
Land cover / imperviousness	ESA WorldCover 10 m, Dynamic World (Sentinel-2), Bhuvan LULC	10 m	Public	OSM landuse
Roads & transit	OpenStreetMap (OSMnx); GTFS for BEST/MTC/DTC where published; Mappls map data	Vector	Public / partner	OSM only
Drainage network	Municipal SWD GIS (BMC BRIMSTOWAD; GCC storm-water drains); OSM waterway=drain/canal; Bhuvan; Survey of India	Vector	Formal request via the PS SPOC / municipal disaster cell; NDA-style access is normal	**Synthetic drain-graph inference (§7.1)**
Traffic speeds	Mappls (MapmyIndia) APIs, TomTom Traffic Flow, Google Maps Platform (Routes/ Distance Matrix with traffic)	Segment, 1–2 min	Developer keys (free tiers exist); Mappls is Indian and government-friendly	Recorded speeds for replay; crowdsourced

Ground truth	Municipal chronic waterlogging registers (BMC publishes yearly lists of a few hundred spots; GCC vulnerable-spot lists), geotagged social media, news photos, IIT-B crowdsourced reports, Sentinel-1 SAR (Copernicus), water-level sensors	Point/polygon	Public + scraping + partner	—
Assets	Hospitals, fire stations, schools, pumping stations, pump inventories, shelters (OSM, municipal lists, DM plans)	Point	Public documents	OSM only

7.1 Synthetic drain-graph inference ("city-in-a-box")

When municipal GIS is unavailable (the normal case at a hackathon), generate a physically plausible network from open data, mark it as inferred, and let Pulse refine it. This follows the synthetic-network generation approaches used in urban drainage research, adapted to Indian design norms.

1. **Skeleton:** take drivable and service roads from OSM; drains follow roads in almost every Indian city.
2. **Outfalls and trunks:** OSM waterway=drain/canal/stream/river + Bhuvan hydrology give nullahs, canals and rivers; mark outfalls where road-following drains meet them; set tidal boundaries at the coast.
3. **Nodes:** place inlets every 30–50 m along road centrelines and at every intersection; add nodes at DEM depressions (priority-flood pits) and at known chronic spots.
4. **Directed edges:** connect consecutive nodes along roads; orient toward decreasing DEM elevation and toward the nearest outfall along the road graph (a shortest-path tree weighted by uphill penalty); resolve flat areas with minimum slope 0.2–0.5%.
5. **Sizing:** compute contributing area per node with the flow-direction raster; design flow by the rational method $Q = C \cdot i \cdot A$ (C from imperviousness, i = design intensity: 25 mm/h legacy, 50 mm/h upgraded corridors); pipe diameter from Manning at full flow, snapped to standard sizes (450, 600, 900, 1200, 1500 mm; box drains for trunks). Invert depth 1–3 m below ground.
6. **Calibration:** adjust sizes so the network's simulated 1-in-2-year response reproduces the municipal chronic-spot list (a coarse but powerful inverse problem); tag every edge with confidence = inferred.
7. **Upgrade path:** when municipal GIS or a SWMM .inp arrives, import it directly and keep only the inferred edges that fill gaps. The system improves without re-architecture.

> Run this script for Chennai live on stage after showing Mumbai: "onboarding a second city takes minutes from open data; the twin then learns its drains from the next monsoon." That single act demonstrates scalability better than any slide.

7.2 Radar image decoding (the hackathon reality)

Until raw volumes arrive, decode the public PNGs: crop to the radar's coverage circle, map legend colours to dBZ classes (lookup table built once per product), georeference by radar coordinates and range (the images are polar-projected with fixed range rings), resample to the city grid, and time-stamp from the image header. Accept that classes are coarse (5 dBZ bins) — the adaptive Z–R and gauge merge compensate. Keep the NetCDF reader ready (`wradlib`, `xarray`, `pyart`) so switching to raw data is a config change.

7.3 Replay engine

Every benchmark event is packaged as a replay bundle: radar frames, gauge series, tide, traffic snapshots (or synthesised anomalies from known flooding times), citizen reports and ground-truth pins with timestamps. The replay service publishes them to the same bus topics at real or accelerated speed (e.g., 30×), so the whole pipeline behaves exactly as live. This is your demo insurance and your continuous validation harness.

8. Technology stack and languages

Polyglot by design — each language where it is strongest, and no language for its own sake. The jury should hear a reason for every choice.

Language	Where it is used	Why
Python 3.12	Sky (pySTEPS, wradlib, xarray), hydrology/DEM (rasterio, rioxarray, WhiteboxTools, RichDEM, GeoPandas, OSMnx), drains (PySWMM, NetworkX/ cuGraph), Flash (PyTorch 2.x + PyTorch Geometric), Pulse (NumPy/JAX EnKF), optimisation (OR-Tools), orchestration (Prefect/Argo), APIs (FastAPI)	The scientific ecosystem lives here; fastest path from equation to running code; GPU access through CuPy/Numba-CUDA and PyTorch.
Rust	Twin hydraulic core (2D stencil kernels via CUDA `cust` or wgpu compute; 1D dynamic-wave solver; inlet coupling), exposed to Python via PyO3/ maturin; Route service (Axum, in-memory graph, time-dependent CH); compiled to WebAssembly for in-browser local what-if simulations	Memory safety with C++ performance; one codebase serves GPU, server and browser. Judges read "Rust + WASM hydrodynamics in the browser" as a signal of engineering depth.
CUDA C++ / Triton	Hand-tuned 2D flux kernels for the city grid (V10); custom fused ops for the surrogate	Peak GPU throughput when Numba/CuPy kernels become the bottleneck.
TypeScript	Command console and public map: Next.js 15 / React 19, MapLibre GL, deck.gl (TerrainLayer, custom water layer), Zustand state, WebSocket live updates; Node BFF; Playwright tests	Type safety across a large UI; the WebGL/deck.gl ecosystem for GPU rendering of depth rasters and 3D water.
WGSL / GLSL	Water rendering shaders (depth-to-colour, animated flow, 3D extrusion over terrain), in-browser compute for local simulation	The "mind-blowing" visual layer that makes 20 cm versus 60 cm legible on a wall screen.
SQL / PL/ pgSQL	PostGIS spatial queries, TimescaleDB hypertables for segment forecasts and observations, pgRouting for V1 routing	Spatial joins, isochrones and time-series in one engine; every product is queryable by an auditor.
Dart (Flutter) or TypeScript (Expo)	Citizen and field app: report-with-photo, offline vector tiles, push alerts, ward-officer pump board	One codebase for Android/iOS; offline-first matters during floods when networks degrade.
Go (optional)	High-throughput ingestion gateway and webhook fan-out if Rust capacity is limited	Simple concurrency for I/O-bound services; interchangeable with Rust here.

YAML / HCL / Bash	Kubernetes manifests, Helm charts, Terraform (AWS Mumbai region / MeghRaj), GitHub Actions CI, DVC pipelines	Reproducible infrastructure; one-command city deployment.

8.1 Frameworks, data and infrastructure

Concern	Choice
Streaming	Redpanda (Kafka-compatible, single binary) for topics; Redis Streams as the V1 stand-in; Protobuf/Avro schemas in a registry.
Orchestration	Argo Workflows on Kubernetes for the 5-min DAG (V10); Prefect for V1; every task idempotent and keyed by run ID.
Storage	PostGIS + TimescaleDB (vectors, time series), MinIO/S3 (rasters as Cloud-Optimised GeoTIFF; ensembles as Zarr; STAC catalogue), Redis (hot state, tile cache), DVC + MLflow (datasets, model registry).
Serving	FastAPI (products, alerts, reports), Rust Axum (routing), TorchServe/Triton or a FastAPI GPU worker (surrogate), TiTiler (dynamic raster tiles), Martin (vector tiles from PostGIS), PMTiles for offline basemaps.
Compute	One GPU node (NVIDIA L4/T4/A10 class or an RTX 4070 workstation) per city for the operational loop; a larger GPU for surrogate training; CPU pool for ingestion and tiles.
Security & governance	Keycloak (OAuth2/OIDC), API keys with per-tenant rate limits, signed CAP messages, audit log of every alert and route, data residency in India, PII-free traffic and anonymised reports, DPDP-Act-aligned consent in the citizen app.
Observability	OpenTelemetry traces across the cycle, Prometheus/Grafana SLOs (latency, mass-balance error, verification scores), alerting on stale feeds.
Standards	OGC API – Features/Tiles, STAC, CAP 1.2, GTFS-Realtime, SWMM .inp, GeoJSON/GeoParquet.

8.2 Monorepo layout

```
varuna/
├── apps/
│   ├── command/            # Next.js ops console + public map (TypeScript, deck.gl, MapLibre)
│   └── field/              # Flutter/Expo citizen & ward-officer app
├── services/
│   ├── ingest/             # Python: radar decode/QC, gauges, tide, NWP, traffic, reports → bus
│   ├── sky/                # Python: QPE, pySTEPS ensembles, NWP blend → rain cube (Zarr)
│   ├── twin/               # Rust core (cuda/wgpu kernels, 1D solver) + PyO3 bindings + Python driver
│   ├── flash/              # PyTorch Geometric surrogate: dataset gen, training, serving
│   ├── pulse/              # EnKF assimilation, traffic-anomaly detector, photo depth CV
│   ├── products/           # segment tables, tiles, hotspots, attribution, CAP alerts, pump MILP
│   ├── route/              # Rust Axum time-dependent routing + reachability + provider feeds
│   └── replay/             # event bundles streamed to the bus at real/accelerated speed
├── city/                   # "city-in-a-box": DEM conditioning, OSM graph, drain synthesis, assets
├── packages/               # shared schemas (Pydantic/Protobuf), geo utils, OpenAPI contracts
├── infra/                  # Terraform, Helm, Argo DAGs, GitHub Actions
├── data/                   # DVC pointers: DEMs, replay bundles, training sets
└── docs/                   # this blueprint, ADRs, verification reports
```

9. API specification and data model

9.1 Core tables (PostGIS / Timescale)

Table	Key columns
road_segment	id, osm_way_id, geom (LineString), class, lanes, oneway, z_min, z_mean, ward_id, exposure_weight
surface_unit	id, geom (Polygon), area, imperviousness, cn, n_manning, depression_depth, cells (array), segment_id
drain_node	id, geom (Point), type, z_ground, z_invert, storage_area, inlet_type, inlet_len, inlet_area, kappa_mean, kappa_var, confidence
drain_edge	id, from_node, to_node, geom, length, shape, diameter, width, height, n_manning, slope, q_full, beta_mean, beta_var, last_desilted, confidence
boundary	id, node_id, type (tide/river/fixed/free), series_source
run	run_id, city, cycle_ts, radar_frame_ts, versions (json), mode (live/replay/degraded), ensemble_n, mass_balance_err
segment_forecast (hypertable)	run_id, segment_id, valid_ts, depth_p10, depth_p50, depth_p90, p_gt_15, p_gt_30, p_gt_45, p_gt_60, safe_until (json per profile)
node_forecast (hypertable)	run_id, node_id, valid_ts, head_p50, p_surcharge, q_surcharge_p50, responsible_edges (json)
observation (hypertable)	id, ts, type (traffic/report/cctv/sensor/sar), geom, value, unit, quality, source, run_id_assimilated
alert	id, run_id, scope (segment/ward/facility), level, cap_xml, sent_channels, acknowledged_by, cleared_ts
pump, pump_assignment	pump id, capacity, location, status; assignment run_id, pump_id, hotspot_id, eta, expected_benefit

9.2 REST endpoints (OpenAPI 3.1; all responses carry run_id and valid_ts)

Endpoint	Purpose
`GET /v1/nowcast/segments?bbox=&t=&profile=`	Depth quantiles, exceedance probabilities and safe-until for road segments in a bounding box at a valid time (GeoJSON / GeoParquet / vector-tile variant at `/tiles/{z}/{x}/{y}.pbf`).
`GET /v1/nowcast/raster?t=&stat=p50|p90|prob30`	Cloud-Optimised GeoTIFF or XYZ tiles of surface depth.
`GET /v1/nowcast/hotspots?ward=&limit=`	Ranked hotspots with time-to-peak, exposure and responsible drains.
`GET /v1/drains/health?bbox=`	Posterior blockage per pipe (mean, variance), inlet clogging, last update — the desilting priority product.
`POST /v1/route` {origin, destination, depart_at, profile, risk_tolerance}	Flood-safe route: geometry, ETA, max expected depth, safe-until, alternates, explanation of avoided segments.
`GET /v1/reachability?facility=&profile=&t=`	Isochrone polygons (5/10/15 min) under the forecast; collapse flag.
`GET /v1/feeds/road-conditions`	Provider feed of impassable/degraded segments with validity windows (GeoJSON, Protobuf) for Mappls/Google-style incident ingestion.
`GET /v1/feeds/gtfs-rt/alerts`	GTFS-Realtime service alerts for affected bus/rail routes.

`GET /v1/alerts?scope=&since=` · `GET /v1/alerts/{id}.cap`	Alert feed and CAP 1.2 documents for Sachet-compatible dissemination.
`POST /v1/reports` {geom, photo, depth_hint, text}	Citizen/field observation ingestion (anonymised, rate-limited, CV depth estimate returned).
`POST /v1/whatif` {rain_scale, cleaned_edges[], pump_plan}	Surrogate-backed scenario run (sub-second): returns deltas versus the operational run.
`GET /v1/runs/{run_id}` · `GET /v1/verification?event=`	Provenance and verification scores (CSI/MAE/Brier) for audit.
`WS /v1/live`	Push of new runs, alerts and reachability changes to consoles.

9.3 Example: route response

```
POST /v1/route
{ "origin": [72.8419, 19.0035], "destination": [72.8628, 19.0176],
  "depart_at": "2019-07-02T17:40:00+05:30", "profile": "ambulance", "risk_tolerance": 0.2 }

200 OK
{ "run_id": "MUM-20190702T174000-sky1.3-twin2.1-flash0.9",
  "route": { "geometry": "<encoded polyline>", "eta_min": 21, "distance_km": 6.8,
             "max_expected_depth_cm": 9, "safe_until": "2019-07-02T18:25:00+05:30" },
  "avoided": [ { "segment_id": 88213, "name": "Hindmata jn", "p_gt_45cm_at_1810": 0.82 },
               { "segment_id": 91007, "name": "Sion rd underpass", "p_gt_60cm_at_1800": 0.71 } ],
  "alternates": [ { "eta_min": 26, "max_expected_depth_cm": 4 } ],
  "confidence": "high (lead 30 min)", "valid_ts": "2019-07-02T17:40:00+05:30" }
```

9.4 CAP alert skeleton

```
<alert xmlns="urn:oasis:names:tc:emergency:cap:1.2">
  <identifier>VARUNA-MUM-20190702T1745-FN-0031</identifier> <sender>varuna@mcgm.gov.in</sender>
  <sent>2019-07-02T17:45:00+05:30</sent> <status>Actual</status> <msgType>Alert</msgType>
<scope>Public</scope>
  <info> <category>Met</category> <event>Street flooding</event> <urgency>Expected</urgency>
    <severity>Severe</severity> <certainty>Likely</certainty>
    <headline>Hindmata junction: water depth likely above 45 cm from 18:20 to 20:00</headline>
    <instruction>Avoid Hindmata and Parel TT; use Ambedkar Rd via Bharatmata. Pumps P-12, P-15
dispatched.</instruction>
    <area> <areaDesc>Ward F/North, Hindmata</areaDesc> <polygon>...</polygon> </area> </info>
</alert>
```

10. Dashboard and experience design

Three audiences, three surfaces, one data model. Design for a control-room wall at 3 a.m. during a cloudburst: high contrast, few colours with fixed meaning, and every number traceable to a run.

10.1 Command console (operators)

- **Map canvas:** MapLibre basemap; deck.gl layers for segment depth (width-scaled lines coloured by p50 depth: <15 cm blue, 15–30 amber, 30–45 orange, >45 red), probability mode (opacity = P(>30 cm)), surcharge markers on manholes, reversed-flow edges in red, drain-health layer (posterior β), and a 3D mode that extrudes water over terrain with an animated flow shader.
- **Time slider:** −60 min (observed) to +180 min (forecast) in 15-min steps; scrubbing re-styles all layers instantly from a preloaded run; ensemble spread band shown under the slider.
- **Right rail:** hotspot list ranked by impact with time-to-peak, "why does this flood" attribution (responsible pipes), ward summaries, alert queue with acknowledge/escalate, pump board with dispatch recommendations and drag-to-assign, reachability clock for hospitals/fire stations.
- **What-if:** sliders for rain scale and tide offset; click pipes to "clean" them; results in under a second from the surrogate, drawn as a diff layer.
- **Mode banner:** live / replay / degraded (which feeds are missing) and the run's verification score on the last comparable event.

10.2 Public map and citizen app

- Simple three-colour street map with "passable until" times; route planner with vehicle selector; push alerts for saved locations; multilingual (English, Hindi, Marathi, Tamil, Telugu, Kannada, Bengali) with icon-first design.
- One-tap report: photo + auto-location + depth chips (ankle/knee/waist); returns "thank you — your report improved the forecast for 3 streets" (it did — Pulse assimilated it).
- Offline-first: PMTiles basemap and last forecast cached; works when the network degrades.

10.3 Partner and machine interfaces

- Navigation providers: road-condition feed; transit: GTFS-RT; emergency dispatch: route API; NDMA/SDMA: CAP; municipal GIS: OGC services; insurers/logistics: segment and hotspot API with SLAs.
- WhatsApp Business bot for ward officers: alert cards with map snapshot, pump orders, and a "report" flow.

10.4 Rendering pipeline

Depth rasters are served as COG tiles (TiTiler) with a colour ramp fixed across runs; segment products as vector tiles (Martin) generated per run and cached in Redis; a WebSocket pushes run IDs so clients swap tile sources atomically. The 3D water layer is a deck.gl custom layer sampling the depth raster in a fragment shader over a TerrainLayer — GPU-side, so a 6-million-cell city renders at 60 fps on a laptop.

11. Build plan: 8-week preparation and the 36-hour finale

> **The rule that decides everything:** nothing scientific is invented during the finale. The 36 hours are for integration, replay-driven validation, UI polish and rehearsal. Every engine must exist as a container with a test before you travel.

11.1 Eight-week preparation (start the week the PS is chosen)

Week	Deliverables	Owner(s) / notes
1	Data requests sent (MoES SPOC for radar volumes and benchmark events; MCGM/GCC disaster cells for gauges and chronic-spot lists; Mappls/TomTom keys; IIT-B contact). DEM, buildings, land cover, OSM downloaded for Mumbai and Chennai. Monorepo, CI, Docker base images, Kubernetes dev cluster (k3d) up. Idea-PPT v1.	Product/Data lead + Infra lead. Replay bundle for 2 July 2019 Mumbai started (radar images scraped from public animations where archived; gauges from BMC reports; ground truth from news/social media).
2	city-in-a-box v1: DEM conditioning at 30 m and 10 m, road graph, synthetic drain graph for Mumbai; chronic-spot register digitised (a few hundred points).	Hydrodynamics + Drain leads. Sanity: DEM depressions vs chronic spots overlap ≥ 60%.
3	Sky v1: radar image decoder, adaptive Z–R, gauge merge, pySTEPS deterministic + 10-member ensemble; verification notebook against gauges for the replay event.	Nowcast lead. Output Zarr rain cube.
4	Twin v1: Numba-CUDA local-inertial 2D at 30 m (city) and 5 m (three hotspot nests); PySWMM 1D on the synthetic graph; coupling with inlet/ surcharge; mass balance test; tide boundary.	Hydrodynamics + Drain leads. First end-to-end replay: rain cube → depth maps.
5	Flash v1: training-set generation (≥ 1,000 runs), GNN surrogate trained; Pulse v1: traffic-anomaly detector on recorded/simulated speeds, report ingestion, EnKF on β for the hotspot nests.	ML lead + Drain lead. Report surrogate RMSE and CSI in docs.
6	Products + Route v1: segment forecast tables, vector tiles, hotspots, safe-until; routing (pgRouting or Valhalla with penalties) for three profiles; reachability isochrones; CAP alert generator; pump MILP.	Backend lead. API contract frozen (OpenAPI).
7	Command v1: console with time slider, probability toggle, hotspot rail, pump board, what-if; public map; WhatsApp alert bot; replay control panel; Chennai onboarding script tested end-to-end.	Frontend lead + Product lead. Internal demo #1.
8	Hardening: degraded modes, latency budget met, verification report for two events, pitch deck and 8-minute demo rehearsed ten times, judge Q&A drilled, laptop + spare GPU box + offline bundles packed.	Everyone. Freeze code 72 h before the finale.

11.2 The 36-hour finale, hour by hour

Hours	Work	Exit criterion

0–2	Set up venue network, GPU box, load replay bundles, bring up all containers, run smoke test end-to-end. Present the running system to mentors in the first mentoring round — momentum matters.	Green smoke test; console shows a live replay cycle.
2–10	Integration pass 1: live IMD radar page ingestion running alongside replay (if it rains anywhere in India, show it); Sky ensemble stable; Twin nests validated against chronic spots; fix data edge cases.	Three consecutive clean cycles at ≤ 3 min each.
10–18	Flash + Pulse in the loop: probability layers, what-if, traffic-anomaly assimilation on replay; drain-health map; attribution panel.	What-if returns in < 1 s; assimilation demonstrably changes a hotspot forecast.
18–26	Route + Command polish: ambulance demo path, reachability clock, WhatsApp alert to a phone, pump board; Chennai one-click onboarding rehearsed on stage hardware; accessibility and language toggles.	Demo script runs start-to-finish without touching a terminal.
26–32	Verification numbers computed for the replay events and put on a slide; README, architecture diagram, API docs; fallback video recorded in case of hardware failure.	Verification table + recorded video exist.
32–36	Rehearsals with timer; assign who answers which judge question; sleep in shifts; final freeze 2 h before judging.	Two clean rehearsals under 8 minutes.

11.3 V1 acceptance checklist (what must be true when the judges arrive)

1. Replay of a Mumbai event streams through the same pipeline as live data; mode banner shows "replay 30×".
2. Sky produces a 20-member 3-hour rain ensemble every cycle and its skill-vs-lead-time chart exists.
3. Twin runs coupled 1D–2D on the city at 30 m and 5 m nests, with visible surcharge at manholes and reversed flow at a tide-locked outfall.
4. Flash returns a 50-member street-level forecast in seconds; probability layer and what-if work.
5. Pulse assimilates at least two observation types on replay and changes the forecast; drain-health map renders.
6. Route returns ambulance/bus/car routes that avoid predicted impassable segments; hospital reachability isochrones shrink over the time slider.
7. Command shows alerts (CAP + WhatsApp), pump dispatch, attribution, and a verification score for the event.
8. Chennai onboarding script runs on stage in minutes and produces a first (uncalibrated) forecast.

12. Team roles and the SIH idea-PPT mapping

12.1 Six roles for six people

Role	Owns	Skills to sharpen now
Nowcast lead (Sky)	Radar decoding/QC, Z–R, gauge merging, pySTEPS ensembles, NWP blend, rain verification	pySTEPS, wradlib, xarray, radar meteorology basics
Hydrodynamics lead (Twin 2D)	DEM conditioning, GPU shallow-water solver, nests, boundaries, mass balance	Numba-CUDA/CuPy, WhiteboxTools, LISFLOOD-FP theory
Drainage & assimilation lead (Graph + Pulse)	Drain synthesis, PySWMM coupling, EnKF, traffic-anomaly detector, drain-health product	SWMM, NetworkX, Bayesian filtering, hydraulics
ML lead (Flash)	Training-set generation, GNN surrogate, uncertainty, attribution, serving	PyTorch Geometric, MeshGraphNets literature, MLflow
Backend & infra lead (Route + platform)	Bus, PostGIS/Timescale, FastAPI, Rust routing, tiles, Kubernetes, CI, security	Rust/Axum, PostGIS, Docker/K8s, OpenAPI
Frontend & product lead (Command + pitch)	Console, public map, field app, WhatsApp bot, demo script, data partnerships, verification report, PPT	Next.js, deck.gl/MapLibre, storytelling, stakeholder outreach

Pairing rule: every engine has a second person who can run it. Nobody's laptop is a single point of failure — everything runs in containers on the shared GPU box.

12.2 Mapping to the SIH idea-submission template

PPT slide	What to put
Idea title	"VARUNA — a self-correcting street-level urban flood nowcasting digital twin (0–3 h) coupling Doppler radar nowcasts, GPU 2D terrain routing and a learning drainage graph, with flood-safe routing for emergency services."
Proposed solution	The six engines in one diagram (Figure 1); the four PS deliverables mapped explicitly; the three differentiators (learning drain, physics-trained surrogate, action products).
Technical approach	The 5-minute cycle (Figure 2), the coupling schematic (Figure 3), the language/stack table, data sources table with access routes.
Feasibility & viability	Compute numbers (§6.3), data fallbacks (§7), replay engine, degraded modes, the 8-week plan, and the V1 acceptance checklist.
Impact & benefits	Lives, gridlock hours, emergency response time, desilting budget targeting; alignment with UFRMP (18 cities), IFLOWS/C-FLOWS complementarity, CAP/Sachet integration.
Research & references	Bates et al. 2010 (local inertial), pySTEPS, SWMM5/PySWMM, MeshGraphNets/GraphCast, NowcastNet/DGMR, EnKF parameter estimation, community-feature flood nowcasting (Sci. Rep. 2023), GPU city-scale forecasting (npj Nat. Hazards 2026), IFLOWS-Mumbai, C-FLOWS, IIT-B Mumbai Flood platform, UFRMP approvals.

13. Demo script: the eight minutes that win

Time	What the jury sees	What you say
0:00	Console on the wall; Mumbai; banner "REPLAY 2 July 2019 · 15:40 · 30×".	"Every monsoon Mumbai knows it will rain. It never knows which junction will drown at 6 pm. This is VARUNA. We are replaying 2 July 2019, three hours before the city stopped."
0:40	Radar animation → rain ensemble fan chart over Hindmata; 20 members diverge after 90 min.	"Radar to rain, gauge-corrected, twenty futures not one. Note how confidence decays after ninety minutes — we show that instead of hiding it."
1:40	Time slider to +2 h: Hindmata, Sion, Milan and Andheri subways turn red; manholes surcharge; a tide-locked outfall shows reversed flow.	"Water enters inlets, travels through pipes with finite capacity, and backs up when the outfall is below high tide. This is a coupled twin, not a heatmap."
2:40	Ground-truth pins appear on the timeline as the replay advances: tweets, photos, BMC log entries at the same spots, 90–120 min after VARUNA flagged them.	"These are the real reports from that evening. VARUNA had them two hours earlier." (Pause. Let it land.)
3:30	Drain X-ray: toggle drain-health; 14 pipes glow red; click one: attribution says it explains most of the Hindmata surcharge; show the EnKF update after a traffic anomaly is assimilated.	"Nobody has a complete drain map, so VARUNA learns it. Traffic jams, citizen photos and sensors are measurements of the drain. The twin gets smarter every flood — and this list is a desilting priority worth crores."
4:30	What-if: drag rain +30%; results in under a second. Clean the 14 pipes; Hindmata drops from 55 cm to 20 cm.	"Physics compiled into a neural network — sub-second, so a control room can ask questions instead of waiting for models."
5:20	Ambulance from KEM to a Sion hospital: naïve route through Hindmata versus VARUNA route; ETA and 'safe until'; hospital reachability isochrone shrinking on the slider; WhatsApp alert buzzes on a phone held up to the jury: "Ward F/N: Hindmata > 45 cm at 18:20. Deploy P-12, P-15 now."	"Prediction becomes action: routes, reachability clocks, CAP alerts that plug into Sachet, and pump dispatch orders."
6:30	Run the Chennai onboarding script; a first forecast appears in minutes.	"City-in-a-box: open data in, digital twin out, drains learned from the next monsoon. Eighteen UFRMP cities, then South Asia."
7:20	Verification slide: CSI, depth MAE, timing error, latency; limitations slide (DEM accuracy, radar access) with mitigations.	"Here is where we are right, where we are wrong, and what a pilot fixes. We built this to sit downstream of IFLOWS and IMD, not to replace them."

Keep a recorded video as fallback; keep the replay speed at 30× so the jury sees hours in seconds; never open a terminal on stage.

14. Judge Q&A: the hard questions, answered

Question	Answer
"Drain GIS is confidential or does not exist. What do you actually model?"	"A synthetic network inferred from roads, DEM and design norms, calibrated to the municipal chronic-spot list, with every pipe carrying a blockage random variable that Pulse estimates from observations. When BRIMSTOWAD or a SWMM model is shared, we import it directly. The system is designed to start ignorant and learn."
"How is this different from IFLOWS?"	"IFLOWS is ward-level, 6–72 h, NWP-driven — the strategic layer. VARUNA is street-level, 0–3 h, radar-driven, with explicit drain hydraulics, learned blockages, probabilistic ensembles and routing. It consumes NCMRWF/IMD fields and IFLOWS outputs as inputs and boundary conditions."
"Where do you get Doppler radar data in real time?"	"Today: IMD's public radar products decoded to reflectivity every 10 minutes, merged with BMC gauges and INSAT-3DS. Pilot: a direct IMD/MoES feed, which the PS sponsor is best placed to enable. The ingestion supports NetCDF/HDF volumes already."
"Your DEM is 30 m with metre-level vertical error. How can you claim centimetre depths?"	"We do not claim centimetre absolute accuracy from a 30 m DEM. We claim correct ponding pattern and timing from terrain plus drains, calibrated at chronic spots against sensors and reports; the pilot budget includes drone LiDAR at ≤ 15 cm vertical accuracy for the flood-prone wards. We report MAE honestly."
"Is a 2D hydrodynamic model really real-time for a whole city?"	"Local-inertial shallow water on a GPU: six million 10 m cells, roughly 5,000 steps for three hours, 15–60 seconds on a single consumer GPU. Ensembles run through the surrogate in seconds. Latency budget is on the slide."
"How do you validate a nowcast without sensors?"	"Replay of past events with time-stamped ground truth from municipal logs, geotagged media, IIT-B reports and Sentinel-1 for large floods; contingency scores per chronic spot; reliability diagrams for probabilities. Sensors improve it, they are not a precondition."
"What about false alarms — closing a road costs money?"	"Probabilities with hysteresis and persistence, operator-set risk tolerance per decision, and calibration curves so a 60% alert means 60%. We optimise for the operator's loss function, not for drama."
"It is December at the finale and Mumbai is dry. What do you demo?"	"The replay engine streams real events through the live pipeline; Chennai's north-east monsoon may give us a live feed too; and the synthetic storm designer runs any IDF-based storm on demand."
"Why a GNN surrogate — why not just run the physics?"	"We do run the physics every cycle. The surrogate is for what physics cannot afford in a 5-minute loop: 50-member probabilities, interactive what-if, and the thousands of evaluations an ensemble Kalman filter needs. The physics keeps generating its training data, so it never drifts far."
"Privacy and legal?"	"Traffic speeds are aggregate, no trajectories; citizen reports are anonymised and consented under the DPDP Act; alerts are signed; data stays in India; no PII leaves the city's tenancy."
"Who pays and how does it scale?"	"UFRMP funds early-warning and data-acquisition components in 18 cities; municipal SaaS per city plus enterprise API for insurers, logistics and mobility; city-in-a-box onboarding in days; the same architecture exports to any monsoon city."
"What breaks first?"	"Radar attenuation in extreme rain and convective initiation — we degrade to gauges/satellite and widen uncertainty. Traffic feed outage — we fall back to reports and sensors. GPU loss — 30 m physics and fewer members. The banner always says which mode we are in."

15. Risk register and honest limitations

Risk	Likelihood	Impact	Mitigation
Raw radar volumes not obtained before finale	High	Medium	Image decoding pipeline; MoES request through the PS SPOC; INSAT/gauge fallback; replay bundles from archived animations and IIT-B/BMC gauges.
No municipal drain GIS	High	Medium	Synthetic drain inference + Pulse learning; SWMM import ready; calibration to chronic-spot registers.
DEM vertical error dominates depth error	High	High	Pattern-and-timing claims; hotspot calibration; LiDAR in pilot; DEM fusion and ICESat-2 correction; state it first.
Surrogate training not converged in time	Medium	Medium	Ship physics-only probabilistic mode (5 members at 30 m); smaller surrogate for nests only.
Traffic API keys/quota	Medium	Low	Recorded speeds for replay; Mappls government-friendly access; reports/sensors alternative.
Venue network or GPU failure	Medium	High	Offline bundles, second GPU laptop, recorded video, CPU degraded mode tested.
Over-scoping during the finale	High	High	V1 checklist is law; feature freeze at hour 26.
Jury perceives competition with MoES systems	Medium	High	Explicit complementarity slide; consume their products; invite co-development.
Nowcast skill collapse beyond 90 min	Certain	Medium	Ensembles + NWP blend; communicate as confidence decay; decisions weighted to 0–90 min.

15.1 Limitations you should state before anyone asks

- Street-level depth accuracy is bounded by terrain data; open DEMs give pattern and timing, LiDAR gives centimetres.
- Radar-based nowcasting cannot foresee convective initiation; beyond ~90 minutes the system is probabilistic by nature.
- Inferred drain networks are hypotheses with uncertainty; the drain-health map is a ranking, not a survey.
- Traffic-as-sensor confounds (accidents, processions) are handled statistically, not perfectly; sensors and CCTV reduce ambiguity.
- Coastal storm-surge coupling uses IMD advisories rather than a full ocean model in V1.

16. Demand, business model and scale

16.1 Why the demand is real now

- **Funded programme:** the Centre has approved the Urban Flood Risk Management Programme for 18 cities — Phase-I for Mumbai, Pune, Hyderabad, Kolkata, Bengaluru, Ahmedabad and Chennai (₹3,075.65 crore) and Phase-II for Bhopal, Bhubaneswar, Guwahati, Jaipur, Kanpur, Patna, Raipur, Thiruvananthapuram, Visakhapatnam, Indore and Lucknow (₹2,444.42 crore). City plans explicitly include flood early-warning systems, real-time monitoring and data acquisition alongside structural drainage works (Bengaluru, for example, earmarked a non-structural component for flood modelling and early warning within its sanctioned outlay).
- **Institutional pull:** MoES built IFLOWS-Mumbai and C-FLOWS and announced similar systems for Bengaluru and Kolkata; IMD is expanding its radar network and nowcasting under Mission Mausam. A street-level, drain-aware, ML-accelerated layer is the natural next procurement.
- **Private demand:** insurers (HDFC ERGO already funds IIT-B's Mumbai flood platform; parametric flood covers need street-level triggers), logistics and mobility platforms (rider safety, ETA reliability), airports, metro and suburban rail, data centres and IT parks, telecom tower operators.
- **Global validation:** Previsico, FloodMapp, 7Analytics and Fathom show councils, utilities and insurers pay for street-level flood nowcasts — none are built for monsoon intensities or Indian data. Jakarta, Dhaka, Manila, Bangkok, Lagos and Ho Chi Minh City share Mumbai's physics.

16.2 Business model

Segment	Offer	Pricing logic (order of magnitude)
Municipal corporations / SDMAs (B2G)	VARUNA City: operational nowcast, console, alerts, pump optimiser, drain-health map, onboarding and LiDAR programme management	Annual SaaS per city in the ₹1–3 crore range, procured under UFRMP/AMRUT 2.0/Smart City SPVs; onboarding fee for data build.
Navigation, mobility, logistics (B2B)	Road-condition feed and route API with SLAs	API subscription by call volume; rider-safety bundles.
Insurance (B2B)	Parametric triggers per segment/parcel, event reconstruction, portfolio risk scoring from the twin	Per-event and per-policy data fees; the twin's replay library is itself a licensable asset.
Infrastructure operators	Airport, metro, rail, ports, telecom: asset-specific reachability and inundation alerts	Enterprise licences.
Public	Free map, alerts and app	Free — it is the trust and data engine (reports feed Pulse).

Operating cost per city during monsoon is dominated by one GPU node plus a small CPU/database footprint — of the order of ₹1–2 lakh per month on an Indian cloud region, scaling down off-season. The margin structure is that of software, not of civil works.

16.3 Impact KPIs to promise a city

- Lead time at chronic spots: from 0 to 90–180 minutes with calibrated probabilities.
- Emergency-service detours avoided and ambulance ETA reliability during events.
- Pump pre-positioning hours saved; reduction in hours above 45 cm at top hotspots.
- Desilting budget targeting: share of desilting spend directed to top-decile drain-health pipes.
- Public reach: alerts delivered via CAP/cell broadcast, app and WhatsApp; citizen reports assimilated.

17. Roadmap: V1 → V10 → V100

Level	Horizon	Capabilities
V1	SIH finale	Mumbai replay + live public radar; coupled Twin at 30 m/5 m; 20-member Sky; Flash for nests or city; Pulse with traffic + reports; Route with three profiles; Command with alerts, pumps, what-if; Chennai one-click onboarding.
V10	6–12 months, one city pilot	Direct IMD radar feed and NCMRWF fields; LiDAR DEM for flood-prone wards; municipal SWMM import; 20–50 IoT level sensors; CCTV water segmentation; Rust/CUDA Twin; nightly surrogate refresh; CAP into Sachet; GTFS-RT with the transit operator; navigation-provider feed live; verification dashboard; ISO-style SLAs.
V10+	12–24 months	All Phase-I UFRMP cities; federated surrogate training across cities; DL nowcast trained on Indian radar archives; SAR assimilation; insurer API; model-predictive scheduling of pumping stations and holding tanks; climate-scenario mode to test where structural drain investments cut flooding most (guide the crores).
V100	3–5 years	A national urban-flood digital twin fabric: every UFRMP city and beyond, integrated with IMD's expanded radar network, NDMA's alerting, and municipal operations; autonomous water operations (gates, pumps, reservoir pre-releases under model-predictive control with human approval); export to South and Southeast Asian megacities; an open standard for street-level flood products that navigation, transit and insurance platforms consume by default.

Appendix A. Equation sheet

Topic	Equation
Reflectivity to rain	$Z = 10^{dBZ/10}$; $R = (Z/a)^{1/b}$; adaptive (a, b) by log-log least squares on gauge pairs; MFB = ΣG/ΣR
Nowcast blend	$R_{blend}(\tau) = e^{-\tau/\tau s} R_{ext} + (1 - e^{-\tau/\tau s}) R_{NWP,bc}$
Effective rain	$R_{eff} = I \cdot \max(0, R - d_s) + (1 - I) \cdot \max(0, R - f)$; Green–Ampt $f = K_s(1 + \psi\Delta\theta/F)$; SCS-CN $Q = (P - 0.2S)^2/(P + 0.8S)$
2D local inertial	$q^{t+\Delta t} = [q^t - g\, h_f\, \Delta t\, \partial(h+z)/\partial x] / [1 + g\, \Delta t\, n^2\, \lvert q^t\rvert / h_f^{10/3}]$; $\Delta t = \alpha\, \Delta x/\sqrt{(g\, h_{max})}$
Pipe capacity	$Q_{full} = (1/n)\, (1-\beta)A \cdot R_h^{2/3}\, S^{1/2}$
Dynamic wave	$\partial A/\partial t + \partial Q/\partial x = 0$; $\partial Q/\partial t + \partial(Q^2/A)/\partial x + gA\partial H/\partial x + gAS_f = 0$; $S_f = n^2 Q\lvert Q\rvert/(A^2 R_h^{4/3})$
Inlet capture	$Q_{inlet} = (1-\kappa) \cdot \min(C_w L\, h^{3/2}, C_o A_o \sqrt{(2gh)}, Q_{avail})$
Surcharge	$Q_{surch} = C_o A_m \sqrt{(2g(H - z_g - h))}$ when $H > z_g + h$
EnKF update	$\theta^a = \theta^f + P_{\theta y}(P_{yy} + R)^{-1}(y + \varepsilon - H(\theta^f))$
Route cost	$c_e(\tau, v) = t_e \cdot \varphi(h_e(\tau))$ if $P(h_e(\tau) > \theta_v) < p_{max}$ else ∞
Pedestrian hazard	unsafe if $h \cdot v \geq 0.5$ m²/s or $h \geq 0.3$ m
Pump dispatch	$\min \Sigma_i\, w_i\, \Sigma_\tau \max(0, V_i(\tau) - \Sigma_p\, x_{p,i}\, c_p\, (\tau - d_{p,i})^+)$, $\Sigma_i\, x_{p,i} \leq 1$

Appendix B. Glossary

Term	Meaning
Nowcast	Very-short-range forecast (0–3 h) built by extrapolating current observations rather than solving atmospheric physics.
DWR / dBZ	Doppler Weather Radar; reflectivity in decibels relative to Z, a measure of returned power related to rain intensity.
Z–R relation	Empirical power law linking reflectivity to rain rate; varies with drop-size distribution.
QPE / QPF	Quantitative precipitation estimation (now) / forecasting (future).
S-PROG / STEPS	Deterministic and stochastic radar extrapolation schemes based on scale decomposition (implemented in pySTEPS).
Local inertial SWE	Simplified 2D shallow-water equations retaining local acceleration; stable and GPU-friendly for floodplain and urban flow.
Dynamic wave	Full 1D Saint-Venant routing in pipes including backwater and surcharge (SWMM's most complete mode).
HGL	Hydraulic grade line — the water level a pipe would show in a standpipe; surcharge occurs when it exceeds ground.
Surcharge / backflow	Pressurised pipe flow forcing water out of manholes; reversal of flow direction when downstream head exceeds upstream head.
EnKF	Ensemble Kalman filter — sequential Bayesian estimation using an ensemble to approximate error covariances.
Surrogate / emulator	Fast learned approximation of a physics model, trained on its simulations.
GNN	Graph neural network — message passing over nodes and edges; natural for pipe networks and terrain adjacency.
CSI / POD / FAR	Critical success index, probability of detection, false-alarm ratio — contingency verification scores.
CAP	Common Alerting Protocol (OASIS), the alert format used by national warning systems including India's Sachet.
GTFS-RT	General Transit Feed Specification – Realtime, for service alerts and vehicle positions.
COG / Zarr / STAC / PMTiles	Cloud-native raster, array, catalogue and tile formats used for scalable geospatial serving.

Appendix C. Links and references

Resource	Where
SIH 2026 problem statements (SIH26085)	sih.gov.in/sih2026PS
IMD radar products and animations	mausam.imd.gov.in (Radar pages; Upper Air / Radar services)
MOSDAC (INSAT-3DS Hydro-Estimator)	mosdac.gov.in
NCMRWF products	ncmrwf.gov.in
INCOIS tide predictions	incois.gov.in
Bhuvan / CartoDEM (NRSC)	bhuvan.nrsc.gov.in
Copernicus GLO-30, FABDEM, ALOS AW3D30	opentopography.org; data.bris.ac.uk (FABDEM); eorc.jaxa.jp
Google Open Buildings; Microsoft building footprints; ESA WorldCover	sites.research.google/open-buildings; github.com/microsoft/GlobalMLBuildingFootprints; esa-worldcover.org
OpenStreetMap / OSMnx	openstreetmap.org; osmnx.readthedocs.io
pySTEPS, wradlib, Py-ART	pysteps.github.io; wradlib.org; arm-doe.github.io/pyart
EPA SWMM5 / PySWMM	epa.gov/water-research/storm-water-management-model-swmm; pyswmm.readthedocs.io
LISFLOOD-FP, SynxFlow/HiPIMS, ItzÏ	Bristol hydrology group; github.com/SynxFlow; itzi.org
PyTorch Geometric; MeshGraphNets/GraphCast papers	pyg.org; DeepMind publications
IIT Bombay Mumbai Flood platform	mumbaiflood.in
IFLOWS-Mumbai / C-FLOWS background	MoES / NCCR / MCGM disaster-management publications
Urban Flood Risk Management Programme (UFRMP)	PIB press releases; NDMA; Lok Sabha replies (July 2026) on Phase-I and Phase-II approvals
OASIS CAP 1.2; NDMA Sachet	docs.oasis-open.org/emergency/cap; sachet.ndma.gov.in
Bates, Horritt & Fewtrell (2010), J. Hydrol. — local inertial formulation	doi.org/10.1016/j.jhydrol.2010.03.027
Farahmand, Xu & Mostafavi (2023), Sci. Rep. — graph deep learning flood nowcasting with community features	doi.org/10.1038/s41598-023-32548-x
GPU-accelerated city-scale urban flood forecasting (2026), npj Natural Hazards	nature.com/articles/s44304-026-00190-y

VARUNA blueprint v1.0 · prepared 2 September 2026 for the SIH 2026 team (VIT) · "Every street. Three hours early."